

FACULTY PROFILE



PARTICULARS	DETAILS	
Name	Rupesh Kumar	
Designation	Lecturer	
Date Of Birth	11/03/1990	
Branch	INFORMATION TECHNOLOGY	
AICTE Unique Id	1-4397574488	
Education Qualification	B.Tech(IT)	
Work Experience:	Teaching	9 Yrs
	Research	NIL
	Industry	NIL
	Others	NIL
Area of Specialization	INFORMATION TECHNOLOGY	
Courses taught (Please Tick)	Diploma	✓
	Post Diploma	NIL
	Under Graduate	NIL
	Post Graduate	NIL
	Post Graduate Diploma	NIL
Research guidance (No. of Students)	Master	NIL
	PhD-	NIL
No. of papers published in	National Journals	NIL
	International Journal	NIL
	Conferences	NIL
Master (Completed /Ongoing)	ONGOING	
PhD (Completed/Ongoing)	NIL	
Projects Carried Out	NIL	
Patents(Filed & Granted)	NIL	
Technology Transfer	NIL	
Research Publications (No. of papers published in National/International/Journals/Conferences)	NIL	
No. of Books published with Details(Name of Book, Publisher with ISBN, Year of Publication etc.)	NIL	

Unit-1.0 Introduction to data structure and Array in 'C'

1.1 Introduction to data structure

1.2 Type of data Structure

1.2.1 Primitive and Non Primitive data Structure

1.2.2 Linear And Non Linear Data Structure

1.3 Declaring and initializing One-Dimensional Array and array Operations Insertion, Searching

1.4 Multi-Dimensional Array and its Operations- Matrix operations

1.5 Drawbacks of Linear Arrays

1.6 Introduction character array and string.

An Introduction to Data Structures

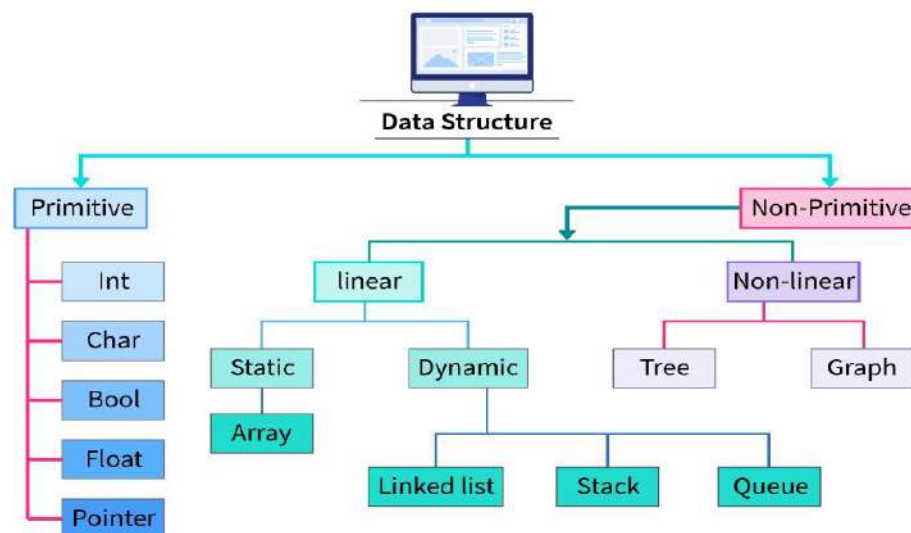
Since the invention of computers, people have been using the term "**Data**" to refer to Computer Information, either transmitted or stored. However, there is data that exists in order types as well. Data can be numbers or texts written on a piece of paper, in the form of bits and bytes stored inside the memory of electronic devices, or facts stored within a person's mind. As the world started modernizing, this data became a significant aspect of everyone's day-to-day life, and various implementations allowed them to store it differently.

Data is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values. The data items are then classified into sub-items, which is the group of items that are not known as the simple primary form of the item.

Data Structure is a branch of Computer Science. The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program. Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required. The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

We can classify Data Structures into two categories:

1. Primitive Data Structure
2. Non-Primitive Data Structure



The following figure shows the different classifications of Data Structures.

Primitive Data Structures

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
4. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -
 1. Linear Data Structures
 2. Non-Linear Data Structures

Linear Data Structures

A data structure that preserves a linear connection among its data elements is known as a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element. However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.

Based on memory allocation, the Linear Data Structures are further classified into two types:

1. **Static Data Structures:** The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compiler time, and their size cannot be changed by the user after being compiled; however, the data stored in them can be altered. The **Array** is the best example of the Static Data Structure as they have a fixed size, and its data can be modified later.
2. **Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time, and their size varies during the run time of the code. Moreover, the user can change the size as well as the data elements stored in these data structures at the run time of the code.

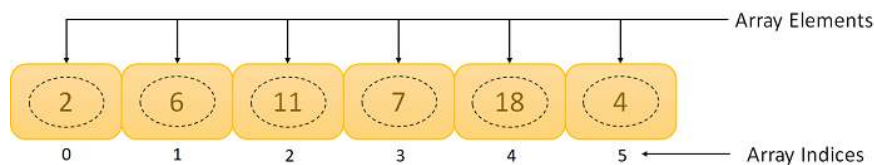
Linked Lists, **Stacks**, and **Queues** are common examples of dynamic data structures

Types of Linear Data Structures

1. Arrays

An **Array** is a data structure used to collect multiple data elements of the same data type into one variable. Instead of storing multiple values of the same data types in separate variable names, we could store all of them together into one variable. This statement doesn't imply that we will have to unite all the values of the same data type in any program into one array of that data type. But there will often be times when some specific variables of the same data types are all related to one another in a way appropriate for an array.

An Array is a list of elements where each element has a unique place in the list. The data elements of the array share the same variable name; however, each carries a different index number called a subscript. We can access any data element from the list with the help of its location in the list. Thus, the key feature of the arrays to understand is that the data is stored in contiguous memory locations, making it possible for the users to traverse through the data elements of the array using their respective indexes.



Arrays can be classified into different types:

1. **One-Dimensional Array:** An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.
2. **Two-Dimensional Array:** An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.
3. **Multidimensional Array:** We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices as per the need.

Some Applications of Array:

1. We can store a list of data elements belonging to the same data type.
2. Array acts as an auxiliary storage for other data structures.
3. The array also helps store data elements of a binary tree of the fixed count.
4. Array also acts as a storage of matrices.

2. Linked Lists

A **Linked List** is another example of a linear data structure used to store a collection of data elements dynamically. Data elements in this data structure are represented by the Nodes, connected using links or pointers. Each node contains two fields, the information field consists of the actual data, and the pointer field consists of the address of the subsequent nodes in the list. The pointer of the last node of the linked list consists of a null pointer, as it points to nothing. Unlike the Arrays, the user can dynamically adjust the size of a Linked List as per the requirements.

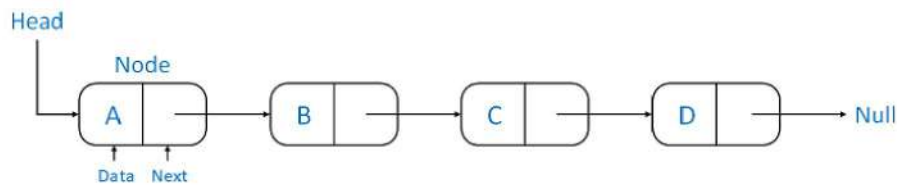


Figure 4. A Linked List

Linked Lists can be classified into different types:

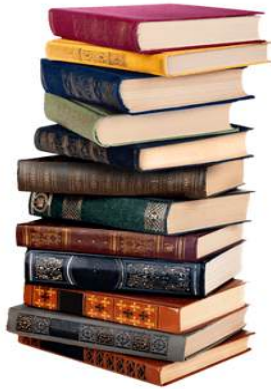
1. **Singly Linked List:** A Singly Linked List is the most common type of Linked List. Each node has data and a pointer field containing an address to the next node.
2. **Doubly Linked List:** A Doubly Linked List consists of an information field and two pointer fields. The information field contains the data. The first pointer field contains an address of the previous node, whereas another pointer field contains a reference to the next node. Thus, we can go in both directions (backward as well as forward).
3. **Circular Linked List:** The Circular Linked List is similar to the Singly Linked List. The only key difference is that the last node contains the address of the first node, forming a circular loop in the Circular Linked List.

Some Applications of Linked Lists:

1. The Linked Lists help us implement stacks, queues, binary trees, and graphs of predefined size.
2. We can also implement Operating System's function for dynamic memory management.
3. Linked Lists also allow polynomial implementation for mathematical operations.
4. We can use Circular Linked List to implement Operating Systems or application functions that Round Robin execution of tasks.
5. Circular Linked List is also helpful in a Slide Show where a user requires to go back to the first slide after the last slide is presented.
6. Doubly Linked List is utilized to implement forward and backward buttons in a browser to move forward and backward in the opened pages of a website.

3. Stacks

A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top. Stacks can be implemented with the help of contiguous memory, an Array, and non-contiguous memory, a Linked List. Real-life examples of Stacks are piles of books, a deck of cards, piles of money, and many more.



The above figure represents the real-life example of a Stack where the operations are performed from one end only, like the insertion and removal of new books from the top of the Stack. It implies that the insertion and deletion in the Stack can be done only from the top of the Stack. We can access only the Stack's tops at any given time.

The primary operations in the Stack are as follows:

1. **Push:** Operation to insert a new element in the Stack is termed as Push Operation.
2. **Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.

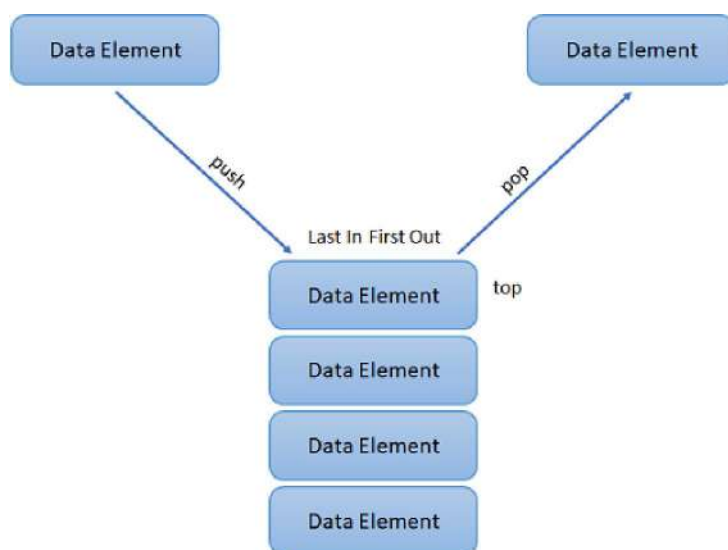


Figure 6. A Stack

Some Applications of Stacks:

1. The Stack is used as a Temporary Storage Structure for recursive operations.
2. Stack is also utilized as Auxiliary Storage Structure for function calls, nested operations, and deferred/postponed functions.
3. We can manage function calls using Stacks.
4. Stacks are also utilized to evaluate the arithmetic expressions in different programming languages.

5. Stacks are also helpful in converting infix expressions to postfix expressions.
6. Stacks allow us to check the expression's syntax in the programming environment.
7. We can match parenthesis using Stacks.
8. Stacks can be used to reverse a String.
9. Stacks are helpful in solving problems based on backtracking.
10. We can use Stacks in depth-first search in graph and tree traversal.
11. Stacks are also used in Operating System functions.
12. Stacks are also used in UNDO and REDO functions in an edit.

4. Queues

A **Queue** is a linear data structure similar to a Stack with some limitations on the insertion and deletion of the elements. The insertion of an element in a Queue is done at one end, and the removal is done at another or opposite end. Thus, we can conclude that the Queue data structure follows FIFO (First In, First Out) principle to manipulate the data elements. Implementation of Queues can be done using Arrays, Linked Lists, or Stacks. Some real-life examples of Queues are a line at the ticket counter, an escalator, a car wash, and many more.



Figure 7. A Real-life Example of Queue

The above image is a real-life illustration of a movie ticket counter that can help us understand the Queue where the customer who comes first is always served first. The customer arriving last will undoubtedly be served last. Both ends of the Queue are open and can execute different operations. Another example is a food court line where the customer is inserted from the rear end while the customer is removed at the front end after providing the service they asked for.

The following are the primary operations of the Queue:

1. **Enqueue:** The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.
2. **Dequeue:** Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.

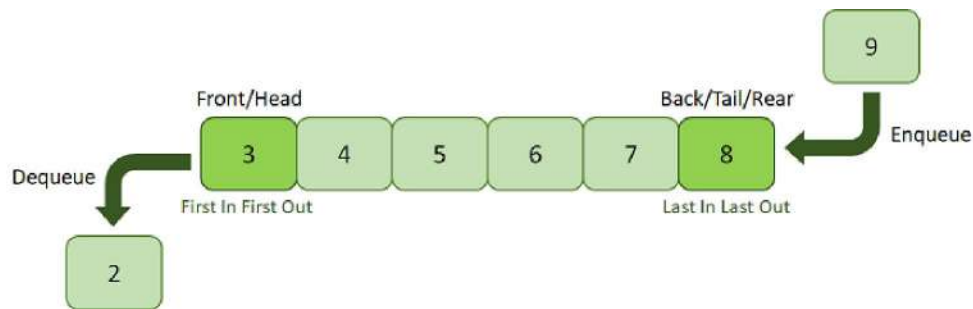


Figure 8. A Queue

Some Applications of Queues:

1. Queues are generally used in the breadth search operation in Graphs.
2. Queues are also used in Job Scheduler Operations of Operating Systems, like a keyboard buffer queue to store the keys pressed by users and a print buffer queue to store the documents printed by the printer.
3. Queues are responsible for CPU scheduling, Job scheduling, and Disk Scheduling.
4. Priority Queues are utilized in file-downloading operations in a browser.
5. Queues are also used to transfer data between peripheral devices and the CPU.
6. Queues are also responsible for handling interrupts generated by the User Applications for the CPU.

Non-Linear Data Structures

Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.

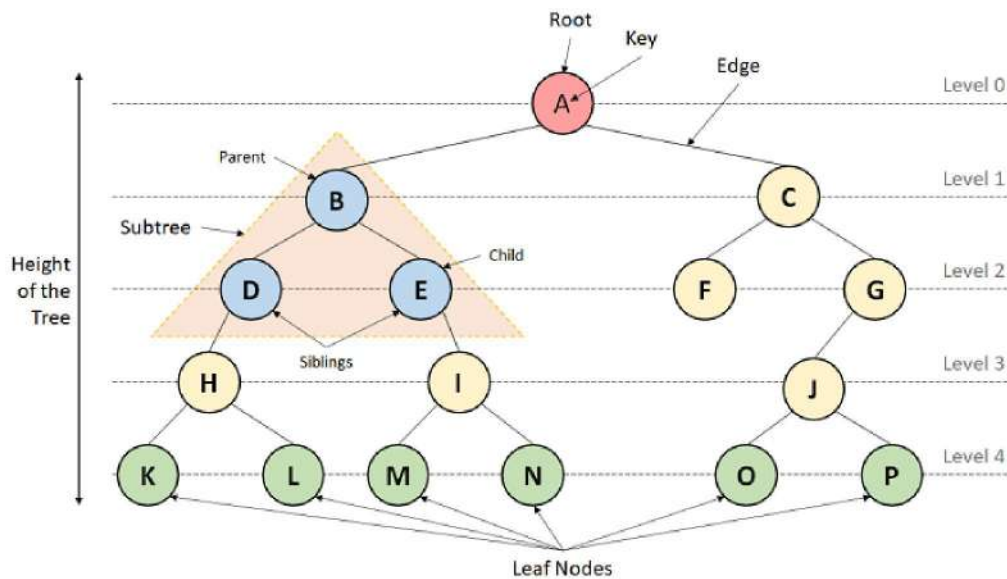
Types of Non-Linear Data Structures

The following is the list of Non-Linear Data Structures that we generally use:

1. Trees

A Tree is a Non-Linear Data Structure and a hierarchy containing a collection of nodes such that each node of the tree stores a value and a list of references to other nodes (the "children").

The Tree data structure is a specialized method to arrange and collect data in the computer to be utilized more effectively. It contains a central node, structural nodes, and sub-nodes connected via edges. We can also say that the tree data structure consists of roots, branches, and leaves connected.



Trees can be classified into different types:

1. **Binary Tree:** A Tree data structure where each parent node can have at most two children is termed a Binary Tree.
2. **Binary Search Tree:** A Binary Search Tree is a Tree data structure where we can easily maintain a sorted list of numbers.
3. **AVL Tree:** An AVL Tree is a self-balancing Binary Search Tree where each node maintains extra information known as a Balance Factor whose value is either -1, 0, or +1.
4. **B-Tree:** A B-Tree is a special type of self-balancing Binary Search Tree where each node consists of multiple keys and can have more than two children.

Some Applications of Trees:

1. Trees implement hierarchical structures in computer systems like directories and file systems.
2. Trees are also used to implement the navigation structure of a website.
3. We can generate code like Huffman's code using Trees.
4. Trees are also helpful in decision-making in Gaming applications.
5. Trees are responsible for implementing priority queues for priority-based OS scheduling functions.
6. Trees are also responsible for parsing expressions and statements in the compilers of different programming languages.
7. We can use Trees to store data keys for indexing for Database Management System (DBMS).
8. Spanning Trees allows us to route decisions in Computer and Communications Networks.

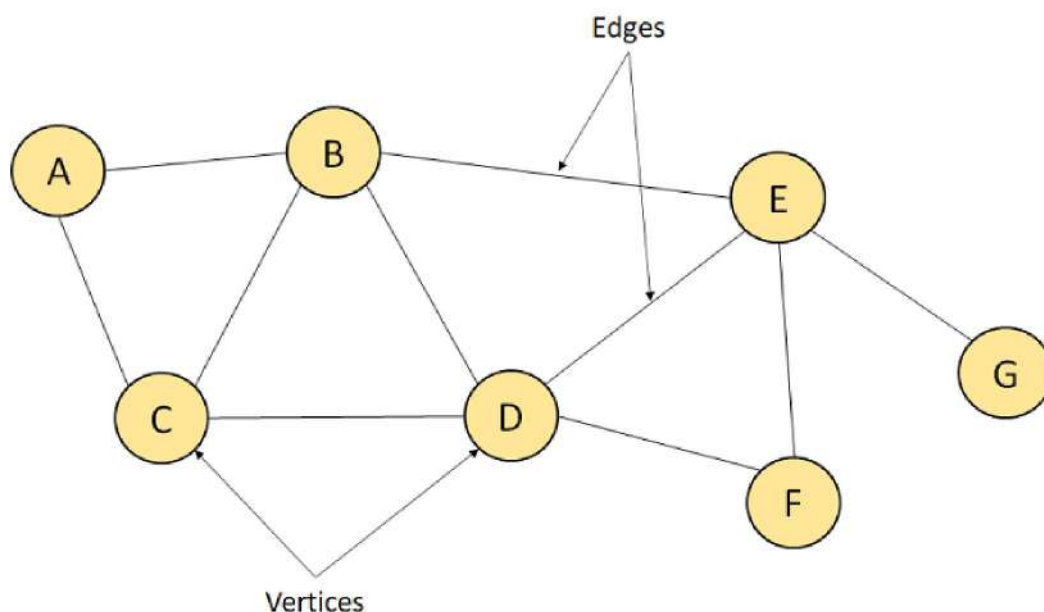
9. Trees are also used in the path-finding algorithm implemented in Artificial Intelligence (AI), Robotics, and Video Games Applications.

2. Graphs

A Graph is another example of a Non-Linear Data Structure comprising a finite number of nodes or vertices and the edges connecting them. The Graphs are utilized to address problems of the real world in which it denotes the problem area as a network such as social networks, circuit networks, and telephone networks. For instance, the nodes or vertices of a Graph can represent a single user in a telephone network, while the edges represent the link between them via telephone.

The Graph data structure, G is considered a mathematical structure comprised of a set of vertices, V and a set of edges, E as shown below:

$$G = (V, E)$$



The above figure represents a Graph having seven vertices A, B, C, D, E, F, G, and ten edges [A, B], [A, C], [B, C], [B, D], [B, E], [C, D], [D, E], [D, F], [E, F], and [E, G].

Depending upon the position of the vertices and edges, the Graphs can be classified into different types:

1. **Null Graph:** A Graph with an empty set of edges is termed a Null Graph.
2. **Trivial Graph:** A Graph having only one vertex is termed a Trivial Graph.
3. **Simple Graph:** A Graph with neither self-loops nor multiple edges is known as a Simple Graph.
4. **Multi Graph:** A Graph is said to be Multi if it consists of multiple edges but no self-loops.

5. **Pseudo Graph:** A Graph with self-loops and multiple edges is termed a Pseudo Graph.
6. **Non-Directed Graph:** A Graph consisting of non-directed edges is known as a Non-Directed Graph.
7. **Directed Graph:** A Graph consisting of the directed edges between the vertices is known as a Directed Graph.
8. **Connected Graph:** A Graph with at least a single path between every pair of vertices is termed a Connected Graph.
9. **Disconnected Graph:** A Graph where there does not exist any path between at least one pair of vertices is termed a Disconnected Graph.
10. **Regular Graph:** A Graph where all vertices have the same degree is termed a Regular Graph.
11. **Complete Graph:** A Graph in which all vertices have an edge between every pair of vertices is known as a Complete Graph.
12. **Cycle Graph:** A Graph is said to be a Cycle if it has at least three vertices and edges that form a cycle.
13. **Cyclic Graph:** A Graph is said to be Cyclic if and only if at least one cycle exists.
14. **Acyclic Graph:** A Graph having zero cycles is termed an Acyclic Graph.
15. **Finite Graph:** A Graph with a finite number of vertices and edges is known as a Finite Graph.
16. **Infinite Graph:** A Graph with an infinite number of vertices and edges is known as an Infinite Graph.
17. **Bipartite Graph:** A Graph where the vertices can be divided into independent sets A and B, and all the vertices of set A should only be connected to the vertices present in set B with some edges is termed a Bipartite Graph.
18. **Planar Graph:** A Graph is said to be a Planar if we can draw it in a single plane with two edges intersecting each other.
19. **Euler Graph:** A Graph is said to be Euler if and only if all the vertices are even degrees.
20. **Hamiltonian Graph:** A Connected Graph consisting of a Hamiltonian circuit is known as a Hamiltonian Graph.

Some Applications of Graphs:

1. Graphs help us represent routes and networks in transportation, travel, and communication applications.
2. Graphs are used to display routes in GPS.
3. Graphs also help us represent the interconnections in social networks and other network-based applications.
4. Graphs are utilized in mapping applications.
5. Graphs are responsible for the representation of user preference in e-commerce applications.
6. Graphs are also used in Utility networks in order to identify the problems posed to local or municipal corporations.
7. Graphs also help to manage the utilization and availability of resources in an organization.

8. Graphs are also used to make document link maps of the websites in order to display the connectivity between the pages through hyperlinks.
9. Graphs are also used in robotic motions and neural networks.

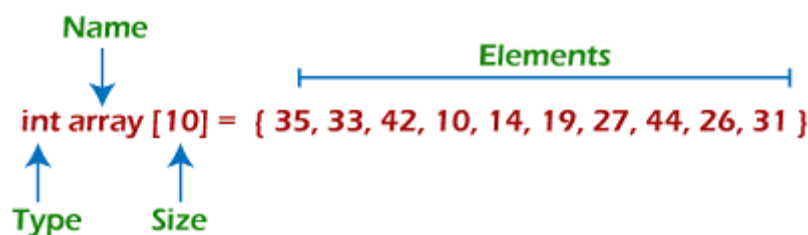
Array in Data Structure

In this article, we will discuss the array in data structure. Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

In C programming, they are the derived data types that can store the primitive type of data such as int, char, double, float, etc. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define a different variable for the marks in different subjects. Instead, we can define an array that can store the marks in each subject at the contiguous memory locations.

Representation of an array

We can represent an array in various ways in different programming languages. As an illustration, let's see the declaration of array in C language -



As per the above illustration, there are some of the following important points -

- Index starts with 0.
- The array's length is 10, which means we can store 10 elements.
- Each element in the array can be accessed via its index.
- n (n - based indexing): The first element of the array can reside at any random index number.

Arrays are good for storing multiple values in a single variable.

C Array Declaration

In C, we have to declare the array like any other variable before using it. We can declare an array by specifying its name, the type of its elements, and the size of its dimensions. When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

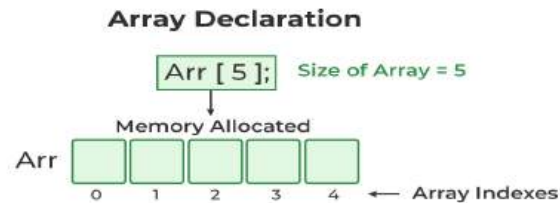
Syntax of Array Declaration

```
data_type array_name [size];
```

or

```
data_type array_name [size1] [size2]...[sizeN];
```

where N is the number of dimensions.



The C arrays are static in nature, i.e., they are allocated memory at the compile time.

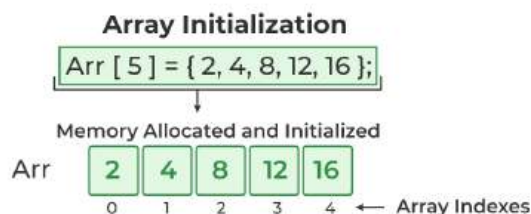
C Array Initialization

Initialization in C is the process to assign some initial value to the variable. When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful value. There are multiple ways in which we can initialize an array in C.

1. Array Initialization with Declaration

In this method, we initialize the array along with its declaration. We use an initializer list to initialize multiple elements of the array. An initializer list is the list of values enclosed within braces `{ }` separated by a comma.

```
data_type array_name [size] = {value1, value2, ... valueN};
```



2. Array Initialization with Declaration without Size

If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases. The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

```
data_type array_name[] = {1,2,3,4,5};
```

The size of the above arrays is 5 which is automatically deduced by the compiler.

Insertion operation

This operation is performed to insert one or more elements into the array. As per the requirements, an element can be added at the beginning, end, or at any index of the array. Now, let's see the implementation of inserting an element into the array.

```

1. #include <stdio.h>
2. int main()
3. {
4.     int arr[20] = { 18, 30, 15, 70, 12 };
5.     int i, x, pos, n = 5;
6.     printf("Array elements before insertion\n");
7.     for (i = 0; i < n; i++)
8.         printf("%d ", arr[i]);
9.     printf("\n");
10.
11.    x = 50; // element to be inserted
12.    pos = 4;
13.    n++;
14.
15.    for (i = n-1; i >= pos; i--)
16.        arr[i] = arr[i - 1];
17.    arr[pos - 1] = x;
18.    printf("Array elements after insertion\n");
19.    for (i = 0; i < n; i++)
20.        printf("%d ", arr[i]);
21.    printf("\n");
22.    return 0;
23. }

```

Output:-

Array elements before insertion

18 30 15 70 12

Array elements after insertion

18 30 15 50 70 12

Search operation

This operation is performed to search an element in the array based on the value or index.

```

1. #include <stdio.h>
2.
3. void main() {
4.     int arr[5] = {18, 30, 15, 70, 12};
5.     int item = 70, i, j=0 ;
6.
7.     printf("Given array elements are :\n");
8.
9.     for(i = 0; i<5; i++) {

```

```

10.    printf("arr[%d] = %d, ", i, arr[i]);
11. }
12.    printf("\nElement to be searched = %d", item);
13.    while( j < 5){
14.        if( arr[j] == item ) {
15.            break;
16.        }
17.
18.        j = j + 1;
19.    }
20.
21.    printf("\nElement %d is found at %d position", item, j+1);
22. }

```

Output

Given array elements are :

Arr[0] = 18 arr[1] = 30 arr[2] = 15 arr[3] = 70 arr[4] = 12

Element to be searched = 70

Element 70 is found at 4 position

Types of Array in C

There are two types of arrays based on the number of dimensions it has. They are as follows:

1. One Dimensional Arrays (1D Array)
2. Multidimensional Arrays

1. One Dimensional Array in C

The One-dimensional arrays, also known as 1-D arrays in C are those arrays that have only one dimension.

Syntax of 1D Array in C

```
array_name [size];
```

1D Array

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Example of 1D Array in C

C

```

// C Program to illustrate the use of 1D array
#include <stdio.h>

int main()
{

    // 1d array declaration
    int arr[5];

    // 1d array initialization using for loop

```



```

for (int i = 0; i < 5; i++) {
    arr[i] = i * i - 2 * i + 1;
}

printf("Elements of Array: ");
// printing 1d array by traversing using for loop
for (int i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}

return 0;
}

```

Output

Elements of Array: 1 0 1 4 9

Array of Characters (Strings)

In C, we store the words, i.e., a sequence of characters in the form of an array of characters terminated by a NULL character. These are called strings in C language.

```

// C Program to illustrate strings
#include <stdio.h>

int main()
{
    // creating array of character
    char arr[6] = { 'G', 'e', 'e', 'k', 's', '\0' };

    // printing string
    int i = 0;
    while (arr[i]) {
        printf("%c", arr[i++]);
    }
    return 0;
}

```

Output

Geeks

To know more about strings, refer to this article – [Strings in C](#)

2. Multidimensional Array in C

Multi-dimensional Arrays in C are those arrays that have more than one dimension. Some of the popular multidimensional arrays are 2D arrays and 3D arrays. We can declare arrays

with more dimensions than 3d arrays but they are avoided as they get very complex and occupy a large amount of space.

A. Two-Dimensional Array in C

A Two-Dimensional array or 2D array in C is an array that has exactly two dimensions. They can be visualized in the form of rows and columns organized in a two-dimensional plane.

Syntax of 2D Array in C

```
array_name[size1] [size2];
```

Here,

- **size1:** Size of the first dimension.
- **size2:** Size of the second dimension.

2D Array

1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

Example of 2D Array in C

C

```
// C Program to illustrate 2d array
#include <stdio.h>

int main()
{
    // declaring and initializing 2d array
    int arr[2][3] = { 10, 20, 30, 40, 50, 60 };

    printf("2D Array:\n");
    // printing 2d array
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
2D Array:
```

```
10 20 30
```

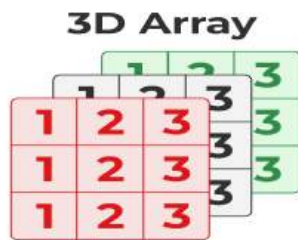
```
40 50 60
```

B. Three-Dimensional Array in C

Another popular form of a multi-dimensional array is Three Dimensional Array or 3D Array. A 3D array has exactly three dimensions. It can be visualized as a collection of 2D arrays stacked on top of each other to create the third dimension.

Syntax of 3D Array in C

```
array_name [size1] [size2] [size3];
```



Example of 3D Array

```
// C Program to illustrate the 3d array
#include <stdio.h>

int main()
{
    // 3D array declaration
    int arr[2][2][2] = { 10, 20, 30, 40, 50, 60 };

    // printing elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                printf("%d ", arr[i][j][k]);
            }
            printf("\n");
        }
        printf("\n\n");
    }
    return 0;
}
```

Output

```
10 20
```

```
30 40
```

```
50 60
```

Character Array

- A Character array is a derived data type in C that is used to store a collection of characters or strings.
- A char data type takes 1 byte of memory, so a character array has the memory of the number of elements in the array. (1* number_of_elements_in_array).
- Each character in a character array has an index that shows the position of the character in the string.
- The first character will be indexed 0 and the successive characters are indexed 1,2,3 etc...
- The null character `\0` is used to find the end of characters in the array and is always stored in the index after the last character or in the last index.

Consider a string "raipur", it is indexed as the following image in a character array.

r	a	i	p	u	r	\0
0	1	2	3	4	5	6(end of string)

Syntax

There are many syntaxes for creating a character array in c. The most basic syntax is,

```
char name[size];
```

- The **name** depicts the name of the character array and **size** is the length of the character array.
- The size can be greater than the length of the string but can not be lesser. If it is lesser then the full string can't be stored and if it is greater the remaining spaces are just left unused.

C Strings

The string can be defined as the one-dimensional array of characters terminated by a null (`'\0'`). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character (`'\0'`) is important in a string since it is the only way to identify where the string ends. When we define a string as `char s[10]`, the character `s[10]` is implicitly initialized with the null in the memory.

There are two ways to declare a string in c language.

1. By char array
2. By string literal

Let's see the example of declaring **string by char array** in C language.

1. **char** ch[6]={r', 'a', 'i', 'p', 'u', 'r'};

Consider a string "raipur", it is indexed as the following image in a character array.

r	a	i	p	u	r	\0
0	1	2	3	4	5	6(end of string)

2. We can also define the **string by the string literal** in C language. For example:

```
char ch[]="javatpoint";
```

String Example in C

Let's see a simple example where a string is declared and being printed. The '%s' is used as a format specifier for the string in c language.

1. #include<stdio.h>
2. #include <string.h>
3. **int** main(){
4. **char** ch[11]={j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};
5. **char** ch2[11]="javatpoint";
- 6.
7. printf("Char Array Value is: %s\n", ch);
8. printf("String Literal Value is: %s\n", ch2);
9. **return** 0;
10. }

Test it Now

Output

```
Char Array Value is: javatpoint
String Literal Value is: javatpoint
```

Different Operations on Matrices

- In this article, we will discuss the following operations on matrices and their properties
 - Matrices Addition
 - Matrices Subtraction
 - Matrices Multiplication

Matrices Addition:

The addition of two matrices $A_{m \times n}$ and $B_{m \times n}$ gives a matrix $C_{m \times n}$. The elements of C are the sum of corresponding elements in A and B which can be shown as:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

```
// C++ Program for matrix addition
#include <iostream>
using namespace std;
int main()
{
    int n = 2, m = 2;
    int a[n][m] = { { 2, 5 }, { 1, 7 } };
    int b[n][m] = { { 3, 7 }, { 2, 9 } };
    int c[n][m];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            cout << c[i][j] << " ";
        cout << endl;
    }
}
```

Output

```
5 12
3 16
```

Matrices Subtraction:

The subtraction of two matrices $A_{m \times n}$ and $B_{m \times n}$ give a matrix $C_{m \times n}$. The elements of C are difference of corresponding elements in A and B which can be represented as:

$$\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

```
// C++ Program for matrix subtraction

#include <iostream>
using namespace std;

int main()
{
    int n = 2, m = 2;
    int a[n][m] = { { 2, 5 }, { 1, 7 } };
    int b[n][m] = { { 3, 7 }, { 2, 9 } };

    int c[n][m];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            c[i][j] = a[i][j] - b[i][j];
        }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            cout << c[i][j] << " ";
        cout << endl;
    }
}
```

Output

```
-1 -2
-1 -2
```

Disadvantages of Array

- Array is homogenous. It means that the elements with similar data type can be stored in it.
- In array, there is static memory allocation that is size of an array cannot be altered.
- There will be wastage of memory if we store less number of elements than the declared size.

Unit 2.0 : Stack and Queue

2.1 Introduction to stack

2.2 Implementation of stack

2.3 Operations on Stack

2.3.1 Push

2.3.2 Pop

2.4 Application of Stack

2.4.1 Expression Evaluation

2.4.2 Infix to Postfix

2.4.3 Infix to Prefix

2.4.4 Recursion

- 2.5 Introduction to Queue
- 2.6 Types of queue
 - 2.6.1 Single Ended Queue
 - 2.6.2 Circular Queue
 - 2.6.3 Double Ended Queue.
- 2.7 Implementation of Queue
- 2.8 Application of Queue

Stack

A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle. Stack has one end, whereas the Queue has two ends (**front and rear**). It contains only one pointer **top pointer** pointing to the topmost element of the stack. Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack. In other words, a ***stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.***

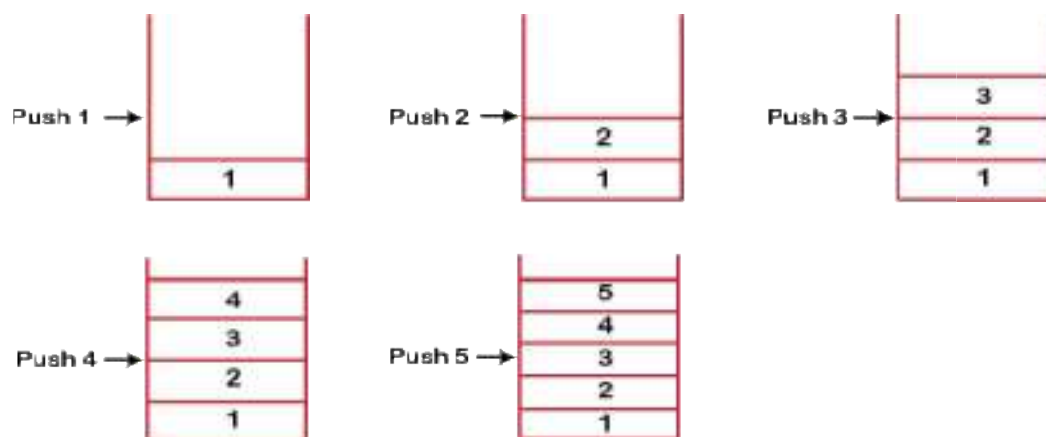
Some key points related to stack

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
 - A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
 - It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.
-

Working of Stack

Stack works on the LIFO pattern. As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.

Suppose we want to store the elements in a stack and let's assume that stack is empty. We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.



Since our stack is full as the size of the stack is 5. In the above cases, we can observe that it goes from the top to the bottom when we were entering the new element in the stack. The stack gets filled up from the bottom to the top.

When we perform the delete operation on the stack, there is only one way for entry and exit as the other end is closed. It follows the LIFO pattern, which means that the value entered first will be removed last. In the above case, the value 5 is entered first, so it will be removed only after the deletion of all the other elements.

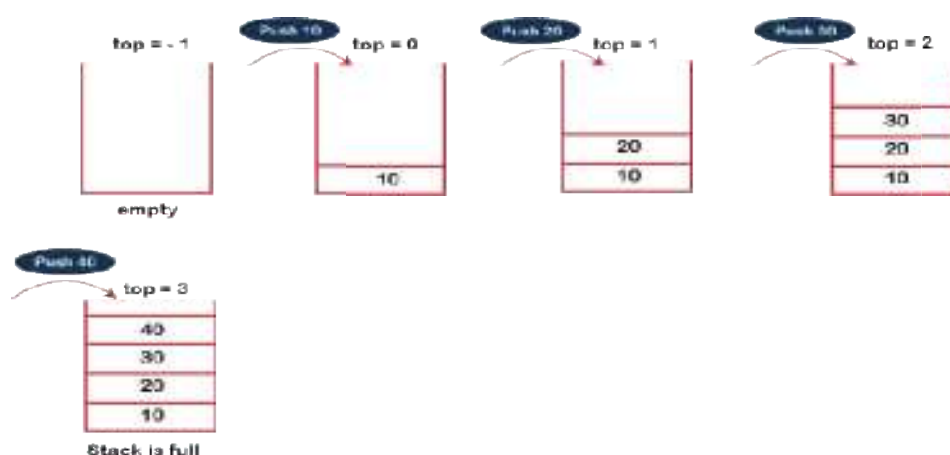
Standard Stack Operations:-

- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.
- **count():** It returns the total number of elements available in a stack.
- **change():** It changes the element at the given position.
- **display():** It prints all the elements available in the stack.

PUSH operation:-

The steps involved in the PUSH operation is given below:

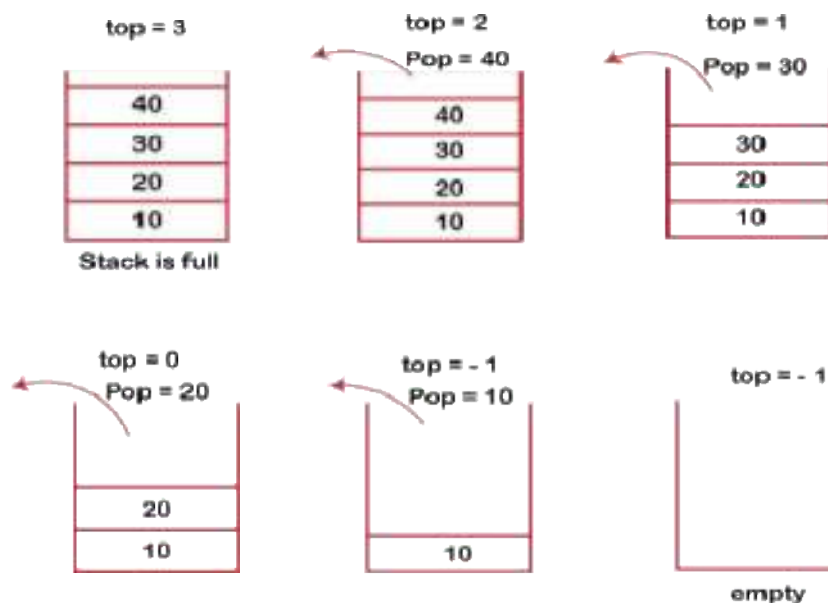
- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the **overflow** condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the **top**.
- The elements will be inserted until we reach the **max** size of the stack.



POP operation:-

The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the ***top***
- Once the pop operation is performed, the top is decremented by 1, i.e., ***top=top-1***.



Applications of Stack

The following are the applications of the stack:

- **Balancing of symbols:** Stack is used for balancing a symbol. For example, we have the following program:

```
1. int main()
2. {
3.   cout<<"Hello";
4.   cout<<"javaTpoint";
5. }
```

As we know, each program has an *opening* and *closing* braces; when the opening braces come, we push the braces in a stack, and when the closing braces appear, we pop the opening braces from the stack. Therefore, the net

value comes out to be zero. If any symbol is left in the stack, it means that some syntax occurs in a program.

- **String reversal:** Stack is also used for reversing a string. For example, we want to reverse a "javaTpoint" string, so we can achieve this with the help of a stack. First, we push all the characters of the string in a stack until we reach the null character. After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.
- **UNDO/REDO:** It can also be used for performing UNDO/REDO operations. For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So, there are three states, a, ab, and abc, which are stored in a stack. There would be two stacks in which one stack shows UNDO state, and the other shows REDO state. If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation.
- **Recursion:** The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.
- **DFS(Depth First Search):** This search is implemented on a Graph, and Graph uses the stack data structure.
- **Backtracking:** Suppose we have to create a path to solve a maze problem. If we are moving in a particular path, and we realize that we come on the wrong way. In order to come at the beginning of the path to create a new path, we have to use the stack data structure.
- **Expression conversion:** Stack can also be used for expression conversion. This is one of the most important applications of stack. The list of the expression conversion is given below:

- *Infix to prefix*
- *Infix to postfix*
- *Prefix to infix*
- *Prefix to postfix*
- *Postfix to infix*

- **Memory management:** The stack manages the memory. The memory is assigned in the contiguous memory blocks. The memory is known as stack memory as all the variables are assigned in a function call stack

memory. The memory size assigned to the program is known to the compiler. When the function is created, all its variables are assigned in the stack memory. When the function completed its execution, all the variables assigned in the stack are released.

Evaluation of Arithmetic Expressions

A stack is a very effective [data structure](#) for evaluating arithmetic expressions in programming languages. An arithmetic expression consists of operands and operators.

In addition to operands and operators, the arithmetic expression may also include parenthesis like "left parenthesis" and "right parenthesis".

Example: $A + (B - C)$

To evaluate the expressions, one needs to be aware of the standard precedence rules for arithmetic expression. The precedence rules for the five basic arithmetic operators are:

Operators	Associativity	Precedence
$^$ exponentiation	Right to left	Highest followed by *Multiplication and /division
*Multiplication, /division	Left to right	Highest followed by + addition and - subtraction
+ addition, - subtraction	Left to right	lowest

Evaluation of Arithmetic Expression requires two steps:

- First, convert the given expression into special notation.
 - Evaluate the expression in this new notation.
-

Notations for Arithmetic Expression

There are three notations to represent an arithmetic expression:

- Infix Notation
 - Prefix Notation
 - Postfix Notation
-

Infix Notation

The infix notation is a convenient way of writing an expression in which each operator is placed between the operands. Infix expressions can be parenthesized or unparenthesized depending upon the problem requirement.

Example: $A + B$, $(C - D)$ etc.

All these expressions are in infix notation because the operator comes between the operands.

Prefix Notation

The prefix notation places the operator before the operands. This notation was introduced by the Polish mathematician and hence often referred to as polish notation.

Example: $+ A B$, $-CD$ etc.

All these expressions are in prefix notation because the operator comes before the operands.

Postfix Notation

The postfix notation places the operator after the operands. This notation is just the reverse of Polish notation and also known as Reverse Polish notation.

Example: $AB +$, $CD+$, etc.

All these expressions are in postfix notation because the operator comes after the operands.

Infix Notation	Prefix Notation	Postfix Notation
$A * B$	$* A B$	AB^*
$(A+B)/C$	$/+ ABC$	$AB+C/$
$(A*B) + (D-C)$	$+*AB - DC$	$AB*DC-+$

What is infix notation?

When the operator is written in between the operands, then it is known as **infix notation**. Operand does not have to be always a constant or a variable; it can also be an expression itself.

For example,

$$(p + q) * (r + s)$$

In the above expression, both the expressions of the multiplication operator are the operands, i.e., **(p + q)**, and **(r + s)** are the operands.

In the above expression, there are three operators. The operands for the first plus operator are p and q, the operands for the second plus operator are r and s. While performing the **operations on the expression, we need to follow some set of rules to evaluate the result**. In the above expression, addition operation would be performed on the two expressions, i.e., p+q and r+s, and then the multiplication operation would be performed.

Syntax of infix notation is given below:

<operand> <operator> <operand>

If there is only one operator in the expression, we do not require applying any rule. For example, $5 + 2$; in this expression, addition operation can be

performed between the two operands (5 and 2), and the result of the operation would be 7.

If there are multiple operators in the expression, then some rule needs to be followed to evaluate the expression.

If the expression is:

$$4 + 6 * 2$$

If the plus operator is evaluated first, then the expression would look like:

$$10 * 2 = 20$$

If the multiplication operator is evaluated first, then the expression would look like:

$$4 + 12 = 16$$

The above problem can be resolved by following the operator precedence rules. In the algebraic expression, the order of the operator precedence is given in the below table:

Operators	Symbols
Parenthesis	(), { }, []
Exponents	^
Multiplication and Division	*, /
Addition and Subtraction	+, -

The first preference is given to the parenthesis; then next preference is given to the exponents. In the case of multiple exponent operators, then the operation will be applied from right to left.

For example:

$$2^{2^3} = 2^8$$

$$= 256$$

After exponent, multiplication, and division operators are evaluated. If both the operators are present in the expression, then the operation will be applied from left to right.

The next preference is given to addition and subtraction. If both the operators are available in the expression, then we go from left to right.

The operators that have the same precedence termed as **operator associativity**. If we go from left to right, then it is known as left-associative. If we go from right to left, then it is known as right-associative.

Problem with infix notation

To evaluate the infix expression, we should know about the **operator precedence** rules, and if the operators have the same precedence, then we should follow the **associativity** rules. The use of parenthesis is very important in infix notation to control the order in which the operation to be performed. Parenthesis improves the readability of the expression. An infix expression is the most common way of writing expression, but it is not easy to parse and evaluate the infix expression without ambiguity. So, mathematicians and logicians studied this problem and discovered two other ways of writing expressions which are prefix and postfix. Both expressions do not require any parenthesis and can be parsed without ambiguity. It does not require operator precedence and associativity rules.

Postfix Expression

The postfix expression is an expression in which the operator is written after the operands. For example, the postfix expression of infix notation (2+3) can be written as 23+.

Some key points regarding the postfix expression are:

- In postfix expression, operations are performed in the order in which they have written from left to right.
 - It does not any require any parenthesis.
 - We do not need to apply operator precedence rules and associativity rules.
-

Conversion of infix to postfix

Here, we will use the stack data structure for the conversion of infix expression to postfix expression. Whenever an operator will encounter, we push operator into the stack. If we encounter an operand, then we append the operand to the expression.

Rules for the conversion from infix to postfix expression

1. Print the operand as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator on to the stack.
3. If the incoming symbol is '(', push it on to the stack.
4. If the incoming symbol is ')', pop the stack and print the operators until the left parenthesis is found.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.
6. If the incoming symbol has lower precedence than the top of the stack, pop and print the top of the stack. Then test the incoming operator against the new top of the stack.
7. If the incoming operator has the same precedence with the top of the stack then use the associativity rules. If the associativity is from left to right then pop and print the top of the stack then push the incoming operator. If the associativity is from right to left then push the incoming operator.
8. At the end of the expression, pop and print all the operators of the stack.

Let's understand through an example.

Infix expression: $K + L - M * N + (O \wedge P) * W / U / V * T + Q$

Input Expression	Stack	Postfix Expression
K		K
+	+	

L	+	K L
-	-	K L +
M	-	K L + M
*	- *	K L + M
N	- *	K L + M N
+	+	K L + M N* K L + M N* -
(	+ (	K L + M N *-
O	+ (	K L + M N* - O
^	+ (^	K L + M N* - O
P	+ (^	K L + M N* - O P
)	+	K L + M N* - O P ^
*	+ *	K L + M N* - O P ^
W	+ *	K L + M N* - O P ^ W
/	+ /	K L + M N* - O P ^ W *
U	+ /	K L + M N* - O P ^ W * U

/	+ /	K L + M N* - O P ^W*U/
V	+ /	KL + MN*-OP^W*U/V
*	+ *	KL+MN*-OP^W*U/V/
T	+ *	KL+MN*-OP^W*U/V/T
+	+	KL+MN*-OP^W*U/V/T* KL+MN*-OP^W*U/V/T*+
Q	+	KL+MN*-OP^W*U/V/T*Q
		KL+MN*-OP^W*U/V/T*+Q+

The final postfix expression of infix expression $(K + L - M * N + (O \wedge P) * W / U / V * T + Q)$ is $KL+MN*-OP^W*U/V/T*+Q+$

Algorithm to evaluate the postfix expression

- Scan the expression from left to right until we encounter any operator.
- Perform the operation
- Replace the expression with its computed value.
- Repeat the steps from 1 to 3 until no more operators exist.

Let's understand the above algorithm through an example.

Infix expression: $2 + 3 * 4$

We will start scanning from the left most of the expression. The multiplication operator is an operator that appears first while scanning from left to right. Now, the expression would be:

Expression = $2 + 34^*$
 $= 2 + 12$

Again, we will scan from left to right, and the expression would be:

Expression = 2 12 +
= 14

Evaluation of postfix expression using stack.

- Scan the expression from left to right.
- If we encounter any operand in the expression, then we push the operand in the stack.
- When we encounter any operator in the expression, then we pop the corresponding operands from the stack.
- When we finish with the scanning of the expression, the final value remains in the stack.

Let's understand the evaluation of postfix expression using stack.

Example 1: Postfix expression: 2 3 4 * +

Input	Stack	
2 3 4 * +	empty	Push 2
3 4 * +	2	Push 3
4 * +	3 2	Push 4
* +	4 3 2	Pop 4 and 3, and perform $4*3 = 12$. Push 12 into the stack.
+	12 2	Pop 12 and 2 from the stack, and perform $12+2 = 14$. Push 14 into the stack.

The result of the above expression is 14.

Example 2: Postfix expression: 3 4 * 2 5 * +

Input	Stack	
3 4 * 2 5 * +	empty	Push 3
4 * 2 5 * +	3	Push 4
* 2 5 * +	4 3	Pop 3 and 4 from the stack and perform $3*4 = 12$. Push 12 into the stack.

Operators	Associativity
-----------	---------------

2 5 * +	12	Push 2
5 * +	2 12	Push 5
* +	5 2 12	Pop 5 and 2 from the stack and perform $5*2 = 10$. Push 10 into the stack.
+	10 12	Pop 10 and 12 from the stack and perform $10+12 = 22$. Push 22 into the stack.

^	Right to Left
*, /	Left to Right
+, -	Left to Right

The result of the above expression is 22.

Algorithm to evaluate postfix expression

1. Read a character
2. If the character is a digit, convert the character into int and push the integer into the stack.
3. If the character is an operator,
 - Pop the elements from the stack twice obtaining two operands.
 - Perform the operation
 - Push the result into the stack.
4. Associativity order
5. **Let's understand the associativity through an example.**

Associativity:- When the operators are having same priority then we go for associativity. It is of two types Left to Right and Right to Left.

Prefix notation?

A prefix notation is another form of expression but it does not require other information such as precedence and associativity, whereas an infix notation requires information of precedence and associativity. It is also known as **polish notation**. In prefix notation, an operator comes before the operands. The syntax of prefix notation is given below:

<operator> <operand> <operand>

For example, if the infix expression is $5+1$, then the prefix expression corresponding to this infix expression is $+51$.

If the infix expression is:

$a * b + c$

↓

$*ab+c$

↓

$+*abc$ (Prefix expression)

Consider another example:

$A + B * C$

First scan: In the above expression, multiplication operator has a higher precedence than the addition operator; the prefix notation of $B*C$ would be $(*BC)$.

$A + *BC$

Second scan: In the second scan, the prefix would be:

$+A *BC$

In the above expression, we use two scans to convert infix to prefix expression. If the expression is complex, then we require a greater number of scans. We need to use that method that requires only one scan, and provides the desired result. If we achieve the desired output through one scan, then the algorithm would be efficient. This is possible only by using a stack.

Conversion of Infix to Prefix using Stack

$K + L - M * N + (O^P) * W/U/V * T + Q$

If we are converting the expression from infix to prefix, we need first to reverse the expression.

The Reverse expression would be:

$$Q + T * V / U / W *) P ^ O (+ N * M - L + K$$

To obtain the prefix expression, we have created a table that consists of three columns, i.e., input expression, stack, and prefix expression. When we encounter any symbol, we simply add it into the prefix expression. If we encounter the operator, we will push it into the stack.

Input expression	Stack	Prefix expression
Q		Q
+	+	Q
T	+	QT
*	+*	QT
V	+*	QTV
/	+*/	QTV
U	+*/	QTVU
/	+*//	QTVU
W	+*//	QTVUW
*	+*//*	QTVUW
)	+*//*)	QTVUW

P	+*//*)	QTVUWP
^	+*//*)^	QTVUWP
O	+*//*)^	QTVUWPO
(	+*//*	QTVUWPO^
+	++	QTVUWPO^*//*
N	++	QTVUWPO^*//*N
*	++*	QTVUWPO^*//*N
M	++*	QTVUWPO^*//*NM
-	++-	QTVUWPO^*//*NM*
L	++-	QTVUWPO^*//*NM*L
+	++-+	QTVUWPO^*//*NM*L
K	++-+	QTVUWPO^*//*NM*LK
		QTVUWPO^*//*NM*LK+- ++

The above expression, i.e., QTVUWPO^*//*NM*LK+-++, is not a final expression. We need to reverse this expression to obtain the prefix expression.

Rules for the conversion of infix to prefix expression:

- First, reverse the infix expression given in the problem.
 - Scan the expression from left to right.
 - Whenever the operands arrive, print them.
 - If the operator arrives and the stack is found to be empty, then simply push the operator into the stack.
 - If the incoming operator has higher precedence than the TOP of the stack, push the incoming operator into the stack.
 - If the incoming operator has the same precedence with a TOP of the stack, push the incoming operator into the stack.
 - If the incoming operator has lower precedence than the TOP of the stack, pop, and print the top of the stack. Test the incoming operator against the top of the stack again and pop the operator from the stack till it finds the operator of a lower precedence or same precedence.
 - If the incoming operator has the same precedence with the top of the stack and the incoming operator is '^', then pop the top of the stack till the condition is true. If the condition is not true, push the '^' operator.
 - When we reach the end of the expression, pop, and print all the operators from the top of the stack.
 - If the operator is ')', then push it into the stack.
 - If the operator is '(', then pop all the operators from the stack till it finds) opening bracket in the stack.
 - If the top of the stack is ')', push the operator on the stack.
 - At the end, reverse the output.
-

Recursion

Recursion is the action of a function calling itself either directly or indirectly, and the associated function is known as a recursive function. A recursive method can be used to tackle some issues with relative ease. Towers of Hanoi (TOH), in order/preorder/postorder tree traversals, DFS of Graph, etc., are a few examples of these issues. By calling a duplicate of itself and resolving the original problem's smaller subproblems, a recursive function solves a specific problem. As and when necessary, many additional recursive calls can be produced. It is crucial to understand that we must present a specific situation to stop this recursion process. Therefore, each time, the function calls itself a simplified version of the initial issue.

What defines the recursive base condition?

The base case solution is given in the recursive program, and the more significant problem's solution is given in terms of more minor issues.

```
1. int fact(int n)
2. {
3.     if (n <= 1) // base case
```

```

4.     return 1;
5.     else
6.         return n*fact(n-1);
7. }

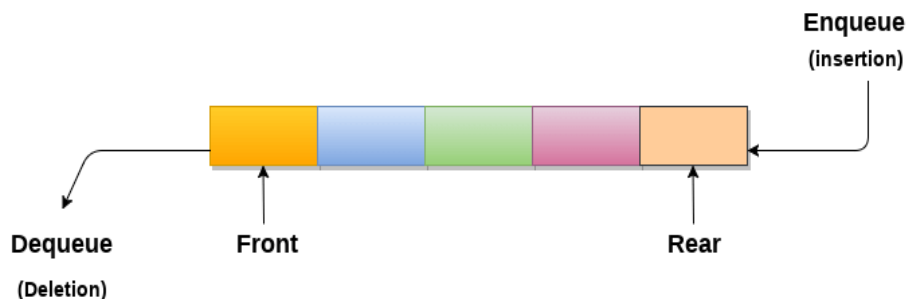
```

Queue

1. A queue can be defined as an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.

2. Queue is referred to be as First In First Out list.

3. For example, people waiting in line for a rail ticket form a queue.



Applications of Queue

Due to the fact that queue performs actions on first in first out basis which is quite fair for the ordering of actions. There are various applications of queues discussed as below.

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
3. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
4. Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
5. Queues are used in operating systems for handling interrupts.

There are different types of queues:

1. Simple Queue/Single ended queue: Simple Queue simply follows FIFO Structure. We can only insert the element at the back and remove the element from the front of the queue.
2. Double-Ended Queue (Deque): In a double-ended queue the insertion and deletion operations, both can be performed from both ends. They are of two types:

- Input Restricted Queue: This is a simple queue. In this type of queue, the input can be taken from only one end but deletion can be done from any of the ends.
 - Output Restricted Queue: This is also a simple queue. In this type of queue, the input can be taken from both ends but deletion can be done from only one end.
3. Circular Queue: This is a special type of queue where the last position is connected back to the first position. Here also the operations are performed in FIFO order.

Basic Operations in Queue Data Structure:

Some of the basic operations for Queue in Data Structure are:

1. Enqueue: Adds (or stores) an element to the end of the queue..
2. Dequeue: Removal of elements from the queue.
3. Peek or front: Acquires the data element available at the front node of the queue without deleting it.
4. rear: This operation returns the element at the rear end without removing it.
5. isFull: Validates if the queue is full.
6. isEmpty: Checks if the queue is empty.

There are a few supporting operations (auxiliary operations):

1. Enqueue Operation in Queue Data Structure:

Enqueue() operation in Queue adds (or stores) an element to the end of the queue.

The following steps should be taken to enqueue (insert) data into a queue:

- Step 1: Check if the queue is full.
- Step 2: If the queue is full, return overflow error and exit.
- Step 3: If the queue is not full, increment the rear pointer to point to the next empty space.
- Step 4: Add the data element to the queue location, where the rear is pointing.
- Step 5: return success.

//ENQUEUE SYNTAX

// Function to add an item to the queue.

// It changes rear and size

```
void enqueue(struct Queue* queue, int item)
```

```
{
```

```
    if (isFull(queue))
```

```
        return;
```

```
    queue->rear = (queue->rear + 1) % queue->capacity;
```

```
    queue->array[queue->rear] = item;
```

```
    queue->size = queue->size + 1;
```

```
    printf("%d enqueued to queue\n", item);
```

```
}
```

2. Dequeue Operation in Queue Data Structure:

Removes (or access) the first element from the queue.

The following steps are taken to perform the dequeue operation:

- Step 1: Check if the queue is empty.
- Step 2: If the queue is empty, return the underflow error and exit.
- Step 3: If the queue is not empty, access the data where the front is pointing.
- Step 4: Increment the front pointer to point to the next available data element.
- Step 5: The Return success.

```
//DEQUEUE OPERATION
// Function to remove an item from queue.
// It changes front and size
int dequeue(struct Queue* queue)
{
    if (isEmpty(queue)) {
        printf("\nQueue is empty\n");
        return;
    }
    int item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size = queue->size - 1;
    return item;
}
```

```
// C program for array implementation of queue
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
```

```
// A structure to represent a queue
struct Queue {
    int front, rear, size;
    unsigned capacity;
    int* array;
};
```

```

// function to create a queue
// of given capacity.
// It initializes size of queue as 0
struct Queue* createQueue(unsigned capacity)
{
    struct Queue* queue
        = (struct Queue*)malloc(sizeof(struct Queue));
    queue->capacity = capacity;
    queue->front = queue->size = 0;

    // This is important, see the enqueue
    queue->rear = capacity - 1;
    queue->array
        = (int*)malloc(queue->capacity * sizeof(int));
    return queue;
}

// Queue is full when size becomes
// equal to the capacity
int isFull(struct Queue* queue)
{
    return (queue->size == queue->capacity);
}

// Queue is empty when size is 0
int isEmpty(struct Queue* queue)
{
    return (queue->size == 0);
}

// Function to add an item to the queue.
// It changes rear and size
void enqueue(struct Queue* queue, int item)
{
    if (isFull(queue))
        return;
    queue->rear = (queue->rear + 1) % queue->capacity;
    queue->array[queue->rear] = item;
    queue->size = queue->size + 1;
    printf("%d enqueued to queue\n", item);
}

// Function to remove an item from queue.
// It changes front and size
int dequeue(struct Queue* queue)
{
    if (isEmpty(queue))

```

```

        return INT_MIN;
    int item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size = queue->size - 1;
    return item;
}

// Function to get front of queue
int front(struct Queue* queue)
{
    if (isEmpty(queue))
        return INT_MIN;
    return queue->array[queue->front];
}

// Function to get rear of queue
int rear(struct Queue* queue)
{
    if (isEmpty(queue))
        return INT_MIN;
    return queue->array[queue->rear];
}

// Driver program to test above functions./
int main()
{
    struct Queue* queue = createQueue(1000);

    enqueue(queue, 10);
    enqueue(queue, 20);
    enqueue(queue, 30);
    enqueue(queue, 40);

    printf("%d dequeued from queue\n", dequeue(queue));
    printf("Front item is %d\n", front(queue));
    printf("Rear item is %d\n", rear(queue));

    return 0;
}

```

Output

```

10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue

```


10 dequeued from queue

Front item is 20

Rear item is 40

Unit 3.0 Pointers, Structure and Union in 'C'

3.1 Introduction and

Features of Pointers

3.2 Declaration of Pointer

3.3 Operations on Pointers

3.4 Array of Pointers

3.5 Pointers to Pointers

3.6 Passing array to

function

3.7 Introduction and

Features of Structure &

Union

3.8 Declaration of Structure

& Union

3.9 Operations on Structure

& Union

3.10 Array of Structure &

Union

3.11 Pointers to Structure

& Union

Pointers

Pointers are one of the core components of the C programming language. A pointer can be used to store the memory address of other variables, functions, or even other pointers. The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.

In this article, we will discuss C pointers in detail, their types, uses, advantages, and disadvantages with examples.

What is a Pointer in C?

A pointer is defined as a derived data type that can store the address of other C variables or a memory location. We can access and manipulate the data stored in that memory location using pointers.

As the pointers in C store the memory addresses, their size is independent of the type of data they are pointing to. This size of pointers in C only depends on the system architecture. If you're looking to master pointers, especially how they work with data structures, the C Programming Course Online with Data Structures offers a comprehensive breakdown of this concept.

Syntax of C Pointers

The syntax of pointers is similar to the variable declaration in C, but we use the (*) dereferencing operator in the pointer declaration.

```
datatype * ptr;
```

where

- ptr is the name of the pointer.
- datatype is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers in C can be divided into three steps:

1. Pointer Declaration
2. Pointer Initialization
3. Pointer Dereferencing

1. Pointer Declaration

In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the (*) dereference operator before its name.

Example

```
int *ptr;
```

The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

2. Pointer Initialization

Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use the (&: ampersand) addressof operator to get the memory address of a variable and then store it in the pointer variable.

Example

```
int var = 10;  
int * ptr;  
ptr = &var;
```

We can also declare and initialize the pointer in a single step. This method is called pointer definition as the pointer is declared and initialized at the same time.

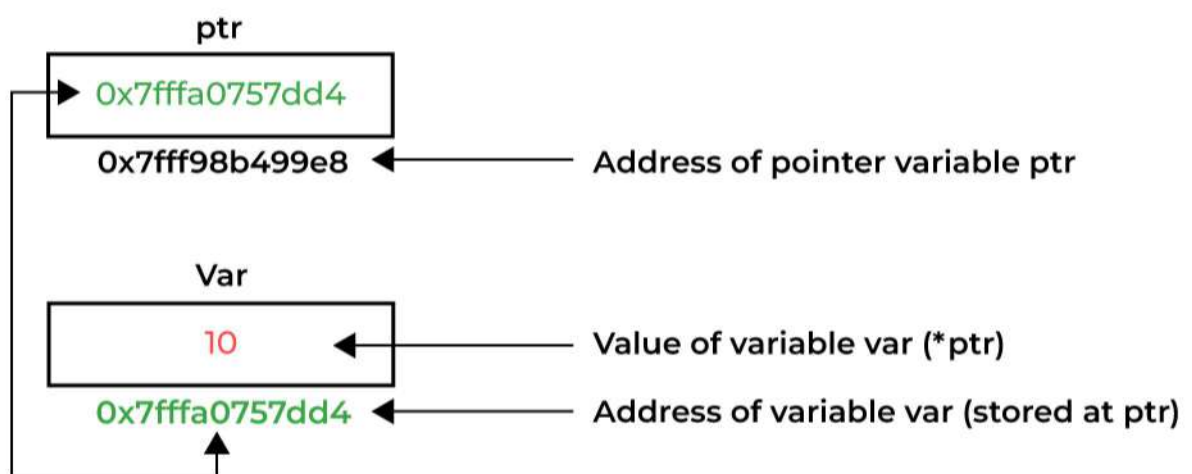
Example

```
int *ptr = &var;
```

Note: It is recommended that the pointers should always be initialized to some value before starting using it. Otherwise, it may lead to number of errors.

3. Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same (*) dereferencing operator that we used in the pointer declaration.



Dereferencing a Pointer in C

Example:-

```
// C program to illustrate Pointers
#include <stdio.h>
void geeks()
{
    int var = 10;
    // declare pointer variable
    int* ptr;
    // note that data type of ptr and var must be same
    ptr = &var;
    // assign the address of a variable to a pointer
    printf("Value at ptr = %p \n", ptr);
    printf("Value at var = %d \n", var);
    printf("Value at *ptr = %d \n", *ptr);
}
// Driver program
int main()
{
    geeks();
    return 0;
}
```

Output

```
Value at ptr = 0x7ffc84068dc
Value at var = 10
Value at *ptr = 10
```

Types of Pointers in C

Pointers in C can be classified into many different types based on the parameter on which we are defining their types. If we consider the type of variable stored in the memory location pointed by the pointer, then the pointers can be classified into the following types:

1. Integer Pointers

As the name suggests, these are the pointers that point to the integer values.

Syntax

```
int *ptr;
```

These pointers are pronounced as Pointer to Integer.

Similarly, a pointer can point to any primitive data type. It can point also point to derived data types such as arrays and user-defined data types such as structures.

2. Array Pointer

Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays. We can create a pointer to an array using the given syntax.

Syntax

```
char *ptr = &array_name;
```

Pointer to Arrays exhibits some interesting properties which we discussed later in this article.

3. Structure Pointer

The pointer pointing to the structure type is called Structure Pointer or Pointer to Structure. It can be declared in the same way as we declare the other primitive data types.

Syntax

```
struct struct_name *ptr;
```

In C, structure pointers are used in data structures such as linked lists, trees, etc.

4. Function Pointers

Function pointers point to the functions. They are different from the rest of the pointers in the sense that instead of pointing to the data, they point to the code. Let's consider a function prototype – `int func (int, char)`, the function pointer for this function will be

Syntax

```
int (*ptr)(int, char);
```

Note: The syntax of the function pointers changes according to the function prototype.

5. Double Pointers

In C language, we can define a pointer that stores the memory address of another pointer. Such pointers are called double-pointers or pointers-to-pointer. Instead of pointing to a data value, they point to another pointer.

Syntax

```
datatype ** pointer_name;
```

Dereferencing Double Pointer

```
*pointer_name; // get the address stored in the inner level pointer  
**pointer_name; // get the value pointed by inner level pointer
```

*Note: In C, we can create multi-level pointers with any number of levels such as –
ptr3, *ptr4, *****ptr5 and so on.*

6. NULL Pointer

The Null Pointers are those pointers that do not point to any memory location. They can be created by assigning a NULL value to the pointer. A pointer of any type can be assigned the NULL value.

Syntax

```
data_type *pointer_name = NULL;  
or  
pointer_name = NULL
```

It is said to be good practice to assign NULL to the pointers currently not in use.

7. Void Pointer

The Void pointers in C are the pointers of type void. It means that they do not have any associated data type. They are also called generic pointers as they can point to any type and can be typecasted to any type.

Syntax

```
void * pointer_name;
```

One of the main properties of void pointers is that they cannot be dereferenced.

C Pointer Arithmetic

The Pointer Arithmetic refers to the legal or valid arithmetic operations that can be performed on a pointer. It is slightly different from the ones that we generally use for mathematical calculations as only a limited set of operations can be performed on pointers. These operations include:

- Increment in a Pointer
- Decrement in a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type
- Comparison of pointers of the same type.
- Assignment of pointers of the same type.

```
//pointer arithmetic  
#include <stdio.h>  
int main()
```

```

{
    // Declare an array
    int v[3] = { 10, 100, 200 };
    // Declare pointer variable
    int* ptr;
    // Assign the address of v[0] to ptr
    ptr = v;

    for (int i = 0; i < 3; i++) {
        // print value at address which is stored in ptr
        printf("Value of *ptr = %d\n", *ptr);
        // print value of ptr
        printf("Value of ptr = %p\n\n", ptr);
        // Increment pointer ptr by 1
        ptr++;
    }
    return 0;
}

```

Output

```

Value of *ptr = 10
Value of ptr = 0x7ffcfe7a77a0
Value of *ptr = 100
Value of ptr = 0x7ffcfe7a77a4
Value of *ptr = 200
Value of ptr = 0x7ffcfe7a77a8

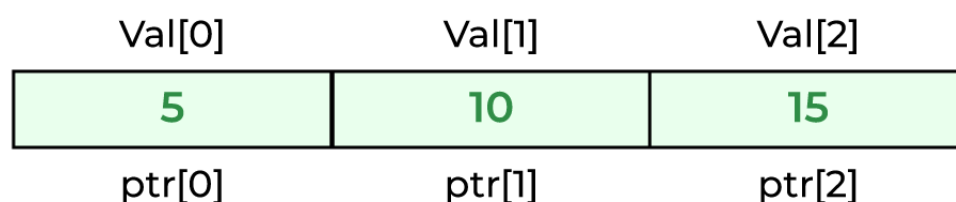
```

C Pointers and Arrays

In C programming language, pointers and arrays are closely related. An array name acts like a pointer constant. The value of this pointer constant is the address of the first element. For example, if we have an array named `val` then `val` and `&val[0]` can be used interchangeably.

If we assign this value to a non-constant pointer of the same type, then we can access the elements of the array using this pointer.

Example 1: Accessing Array Elements using Pointer with Array Subscript



```
// C Program to access array elements using pointer
#include <stdio.h>
```

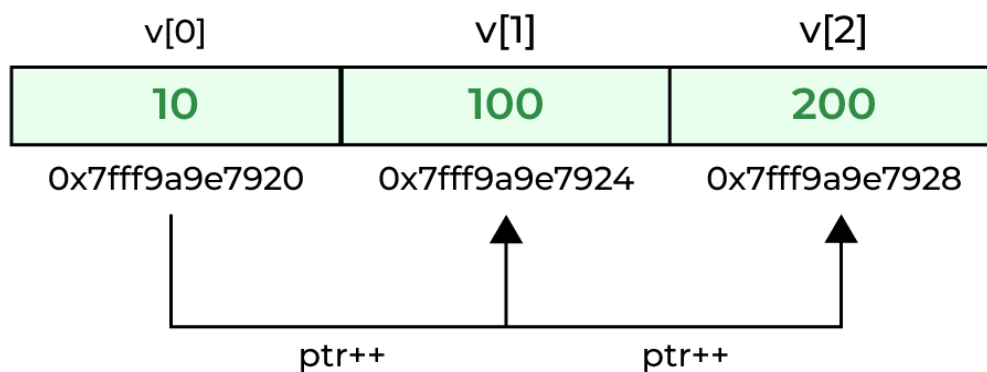
```
void geeks()
{
    // Declare an array
    int val[3] = { 5, 10, 15 };
    // Declare pointer variable
    int* ptr;
    // Assign address of val[0] to ptr.
    // We can use ptr=&val[0];(both are same)
    ptr = val;
    printf("Elements of the array are: ");
    printf("%d, %d, %d", ptr[0], ptr[1], ptr[2]);
    return;
}
// Driver program
int main()
{
    geeks();
    return 0;
}
```

Output

Elements of the array are: 5, 10, 15

Not only that, as the array elements are stored continuously, we can pointer arithmetic operations such as increment, decrement, addition, and subtraction of integers on pointer to move between array elements.

Example 2: Accessing Array Elements using Pointer Arithmetic



```
// C Program to access array elements using pointers
```



```

#include <stdio.h>
int main()
{
    // defining array
    int arr[5] = { 1, 2, 3, 4, 5 };
    // defining the pointer to array
    int* ptr_arr = arr;
    // traversing array using pointer arithmetic
    for (int i = 0; i < 5; i++) {
        printf("%d ", *ptr_arr++);
    }
    return 0;
}

```

Output

```
1 2 3 4 5
```

This concept is not limited to the one-dimensional array, we can refer to a multidimensional array element as well using pointers.

Uses of Pointers in C

The C pointer is a very powerful tool that is widely used in C programming to perform various useful operations. It is used to achieve the following functionalities in C:

1. Pass Arguments by Reference
2. Accessing Array Elements
3. Return Multiple Values from Function
4. Dynamic Memory Allocation
5. Implementing Data Structures
6. In System-Level Programming where memory addresses are useful.
7. In locating the exact value at some memory location.
8. To avoid compiler confusion for the same variable name.
9. To use in Control Tables.

Advantages of Pointers

Following are the major advantages of pointers in C:

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or a structure can be accessed efficiently with pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce the length of the program and its execution time as well.

Disadvantages of Pointers

Pointers are vulnerable to errors and have following disadvantages:

- Memory corruption can occur if an incorrect value is provided to pointers.
- Pointers are a little bit complex to understand.
- Pointers are majorly responsible for memory leaks in C.
- Pointers are comparatively slower than variables in C.
- Uninitialized pointers might cause a segmentation fault.

Array of Pointers in C

In C, a pointer array is a homogeneous collection of indexed pointer variables that are references to a memory location. It is generally used in C Programming when we want to point at multiple memory locations of a similar data type in our C program. We can access the data by dereferencing the pointer pointing to it.

Syntax:

```
pointer_type *array_name [array_size];
```

Here,

- `pointer_type`: Type of data the pointer is pointing to.
- `array_name`: Name of the array of pointers.
- `array_size`: Size of the array of pointers.

Note: It is important to keep in mind the operator precedence and associativity in the array of pointers declarations of different type as a single change will mean the whole different thing. For example, enclosing `*array_name` in the parenthesis will mean that `array_name` is a pointer to an array.

Example:

```
// C program to demonstrate the use of array of pointers
#include <stdio.h>
```

```
int main()
{
    // declaring some temp variables
    int var1 = 10;
    int var2 = 20;
    int var3 = 30;

    // array of pointers to integers
    int* ptr_arr[3] = { &var1, &var2, &var3 };

    // traversing using loop
    for (int i = 0; i < 3; i++) {
        printf("Value of var%d: %d\tAddress: %p\n", i + 1, *ptr_arr[i], ptr_arr[i]);
    }
}
```

```

    }

    return 0;
}

```

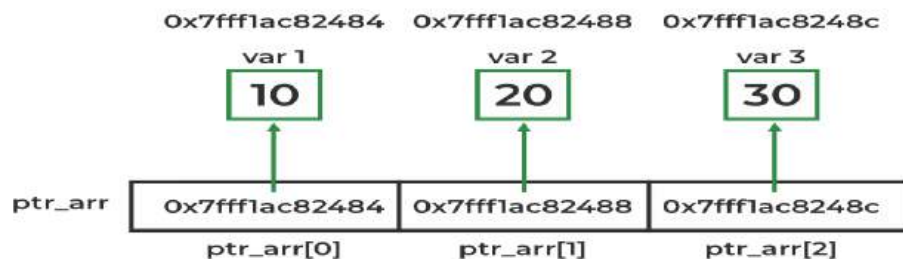
Output

```

Value of var1: 10   Address: 0x7fff1ac82484
Value of var2: 20   Address: 0x7fff1ac82488
Value of var3: 30   Address: 0x7fff1ac8248c

```

Explanation:



As shown in the above example, each element of the array is a pointer pointing to an integer. We can access the value of these integers by first selecting the array element and then dereferencing it to get the value.

Passing array to the function

In C, the whole array cannot be passed as an argument to a function. However, you can pass a pointer to an array without an index by specifying the array's name.

Arrays in C are always passed to the function as pointers pointing to the first element of the array.

Syntax

In C, we have three ways to pass an array as a parameter to the function. In the function definition, use the following syntax:

```
return_type foo ( array_type array_name[size], ...);
```

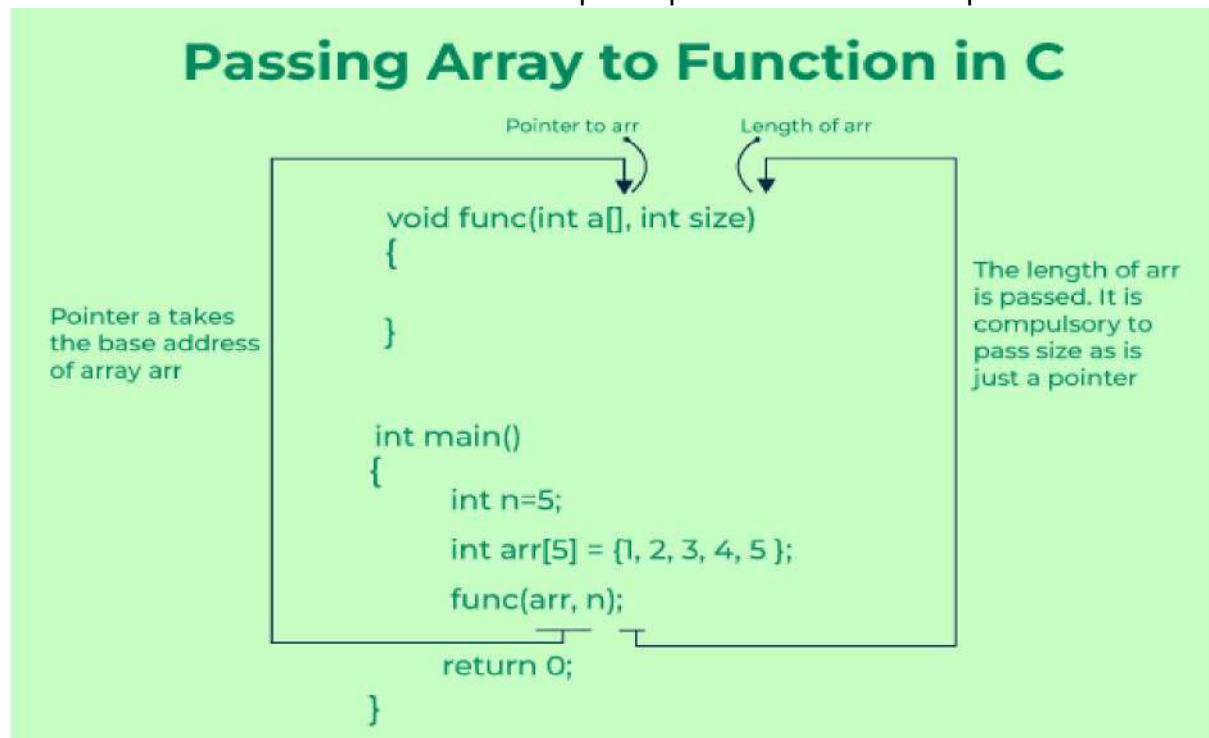
Mentioning the size of the array is optional. So the syntax can be written as:

```
return_type foo ( array_type array_name[], ...);
```

In both of the above syntax, even though we are defining the argument as array it will still be passed as a pointer. So we can also write the syntax as:

```
return_type foo ( array_type* array_name, ...);
```

But passing an array to function results in [array decay](#) due to which the array loses information about its size. It means that the size of the array or the number of elements of the array cannot be determined anymore. To explore how functions and arrays work together in complex data structures, the C Programming Course Online with Data Structures offers in-depth explanations and examples.



Examples of Passing Array to Function in C

Example 1: Checking Size After Passing Array as Parameter

// C program to pass the array as a function and check its size

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

// Note that arr[] for fun is just a pointer even if square

// brackets are used. It is same as

// void fun(int *arr) or void fun(int arr[size])

```
void func(int arr[8])
```

```
{
```

```
    printf("Size of arr[] in func(): %d bytes",
           sizeof(arr));
```

```
}
```

// Drive code

```
int main()
```

```
{
```

```
    int arr[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
```

```
printf("Size of arr[] in main(): %dbytes\n",
      sizeof(arr));
func(arr);

return 0;
}
```

Output

```
Size of arr[] in main(): 32bytes
Size of arr[] in func(): 8 bytes
```

Unit 4.0: Linked List

4.1 Introduction of Linked List

4.1.1 node,

4.1.2 next address

4.1.3 pointer,

4.1.4 null pointer

4.2 Types of Linked List

4.2.1 Singly Linked List

4.2.2 Doubly Linked List

4.2.3 Circular Linked List

4.3 Operations on Linked List

4.3.1 Searching

4.3.2 insertion

4.3.3 Deletion

4.4 Applications of Linked List

Linked List

A linked list is a fundamental data structure in computer science. It mainly allows efficient insertion and deletion operations compared to arrays. Like arrays, it is also used to implement other data structures like stack, queue and deque.

A linked list is a type of linear data structure individual items are not necessarily at contiguous locations. The individual items are called nodes and connected with each other using links.

- A node contains two things first is data and second is a link that connects it with another node.
- The first node is called the head node and we can traverse the whole list using this head and next links.

Linked List is a linear data structure, in which elements are not stored at a contiguous location, rather they are linked using pointers. Linked List forms a series

of connected nodes, where each node stores the data and the address of the next node.

Node Structure: A node in a linked list typically consists of two components:

1. **Data:** It holds the actual value or data associated with the node.
2. **Next Pointer or Reference :** It stores the memory address (reference) of the next node in the sequence.

Head and Tail: The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or nullptr, indicating the end of the list. This node is known as the tail node.

Operations on Linked Lists

1. **Insertion:** Adding a new node to a linked list involves adjusting the pointers of the existing nodes to maintain the proper sequence. Insertion can be performed at the beginning, end, or any position within the list
2. **Deletion:** Removing a node from a linked list requires adjusting the pointers of the neighboring nodes to bridge the gap left by the deleted node. Deletion can be performed at the beginning, end, or any position within the list.
3. **Searching:** Searching for a specific value in a linked list involves traversing the list from the head node until the value is found or the end of the list is reached.

```

int main()
{
construct c1; //creating an object of construct
construct c2(5,6);
return 0;
}

```

An example of parameterised constructor

```

// CPP program to illustrate
// parameterized constructors
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }

    int getX() { return x; }
    int getY() { return y; }
};

```

```

int main()
{
    // Constructor called
    Point p1(10, 15);

    // Access values assigned by constructor
    cout << "p1.x = " << p1.getX()
        << ", p1.y = " << p1.getY();

    return 0;
}

```

Uses of Parameterized constructor:

1. It is used to initialize the various data elements of different objects with different values when they are created.

Server Virtualisation: Server virtualization is a process of partitioning the resources of a single server into multiple virtual servers. These virtual servers can run as separate machines. Server virtualization allows businesses to run multiple independent tasks with different configurations using a single (host) server. The process also saves the hardware cost involved in keeping a host of physical servers.

Block and File level Storage Virtualisation: Storage virtualization is a series of servers that are managed by a virtual storage system. The servers aren't aware of exactly where their data is stored. This technology manages the storage of data from multiple users and utilized as a single storage system. storage virtualization software maintains smooth operations and consistent performance despite changes, break down and differences in the connected equipment.

Data virtualization: This is a kind of virtualization in which the data is collected from various sources and managed that in a single server without knowing more about the technical information like how data is collected, stored & formatted. Then the stored data can be arranged in such a way so that its virtual view can be accessed by interested users by using the various cloud services remotely. Many big giant companies are providing their services like Oracle, IBM, At scale, Cdata etc.

Desktop as a service (DaaS)

Desktop as a service (DaaS) is a cloud-based desktop virtualization service hosted by a third party enterprise. The third party cloud provider manages all back-end resources, such as desktop storage, compute, and networking, including the virtual cloud machines that run the desktop operating systems. The provider streams the virtual desktops to end-user devices, allowing anytime, anywhere access to desktops and applications. There are two kinds of desktops available in DaaS. These are:

1. **Persistent desktop:** Users have the ability to customize and save the desktop so that it will look the same way each time when a particular user logs in. Persistent desktop requires more storage than non-persistent desktop.
2. **Non-persistent desktop:** Desktops are wiped each time the user logs out—they are merely a way to access shared cloud services.

DaaS Advantages:

- Easy platform migration
- Total cost reduction
- Minimized complexity
- Disaster recovery
- Uninterrupted connectivity
- Increased performance
- Personalization
- Reliability
- Data security